Extending the R-Vim Workflow: LaTeX Integration and Dynamic Snippets

Ronald 'Ryy' G. Thomas

2026-05-17

Table of contents

2026-05-17 16:55 PDT

Setting the stage for a unified editing environment.

1 Introduction

I did not really know how to connect Vim, R, and LaTeX into a single productive workflow until I spent a weekend configuring vimtex and UltiSnips from scratch. Before that weekend, my workflow was fragmented: I wrote R code in one editor, switched to another for LaTeX, and lost time every time I moved between tools.

The frustration peaked when I realised I was spending more time switching contexts than actually writing code or prose. Every tool had its own shortcuts, its own conventions, and its own mental model. I needed a single environment that could handle R scripts, R Markdown documents, and LaTeX files without forcing me to leave Vim.

We document the complete configuration arrived at after several rounds of trial and error, then extend it with a technique discovered later: embedding Python code directly inside UltiSnips

snippets. Static snippets cover predictable boilerplate. Python-interpolated snippets cover cases where the expansion must be computed at runtime (for instance, generating a variable number of repeated elements based on a value typed into the first tabstop).

More formally, we document the Vim-plugins layer (Layer 6) of the Workflow Construct described in post 52, specifically the R-plus-LaTeX subset. The plugins documented here (Nvim-R or zzvim-r, vimtex, UltiSnips) compose to give Vim the same edit-run-inspect cycle that RStudio provides for R and Skim provides for LaTeX, without leaving the editor. They are the operative artefacts that make the Editor layer (post 26) usable for the applied biostatistician's daily R-and-LaTeX work.

1.1 Motivations

- I was tired of context-switching between RStudio, a LaTeX editor, and the terminal every time I worked on a statistical report.
- I wanted a single editor that could send R code to a live REPL, compile LaTeX documents, and expand code snippets without leaving the keyboard.
- My existing Vim setup had no awareness of R or LaTeX filetypes, so I had to configure everything from scratch.
- I needed ALE linting and auto-fixing for R code style consistency across projects.
- I wanted UltiSnips templates for R Markdown YAML headers so I could scaffold new analysis files in seconds.
- I had never configured ftplugin files before and wanted to understand how Vim dispatches settings by filetype.
- Static snippets proved too rigid for cases that require a variable number of output elements; I needed a way to compute expansion content at the moment of triggering.

1.2 Objectives

1. Install and configure vimtex, UltiSnips, ALE, and supporting plugins in a single .vimrc file.
2. Create filetype-specific configurations for R and R Markdown that open an R terminal and define useful keybindings.
3. Set up UltiSnips snippet expansion and popup menu navigation without conflicts between Tab mappings.
4. Walk through a practical example of editing an R Markdown file with the Palmer Penguins dataset inside this configured Vim environment.
5. Extend UltiSnips with Python interpolation to generate dynamic, parametric snippet expansions.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

The kind of focused environment where configuration work happens best.

2 Prerequisites and Setup

Before proceeding, the following software must be installed on the macOS system:

- **Vim 8.1+ compiled with Python 3 support** or **Neovim 0.5+** with terminal support. To verify Python 3 support, run `:echo has('python3')` inside Vim; a return value of `1` confirms it is available.
- **R 4.4+** from CRAN
- **MacTeX** from TUG
- **vim-plug** plugin manager, installed with:

```
curl -fLo ~/.vim/autoload/plug.vim \
  --create-dirs \
  https://raw.githubusercontent.com/\
junegunn/vim-plug/master/plug.vim
```

A working terminal emulator (iTerm2 or Kitty) and basic familiarity with Vim normal and insert modes are also assumed. See the companion post 'Setting up a minimal neovim…' for details on installing plugins with Neovim. Python 3 support is required for UltiSnips to function at all; without it, the plugin loads but snippet expansion silently fails.

3 What is a Vim-Based R and LaTeX Workflow?

A Vim-based R and LaTeX workflow is a configuration that supports writing R code, R Markdown documents, and LaTeX files inside a single Vim session. Instead of switching between RStudio for data analysis and a dedicated LaTeX editor for typesetting, Vim serves as the unified interface.

Think of it as turning Vim into a lightweight IDE. The .vimrc file loads plugins for LaTeX compilation (vimtex), code snippet expansion (UltiSnips), and syntax linting (ALE). Filetype plugins detect whether the open file is an R script or an R Markdown document and automatically open an R terminal in a split pane. Keybindings allow sending code to the REPL, inspecting data objects, and rendering documents without leaving the editor.

The practical benefit is speed. Once configured, one can move from opening a file to running a complete analysis and compiling a PDF report without touching the mouse or switching applications.

4 Getting Started

The first step is to create or edit the `~/.vimrc` file with the full plugin configuration. The configuration below is a complete, working .vimrc that I use daily. It includes plugin declarations, global settings, and keybindings.

After saving the .vimrc, open Vim and run `:PlugInstall` to download and install all declared plugins.

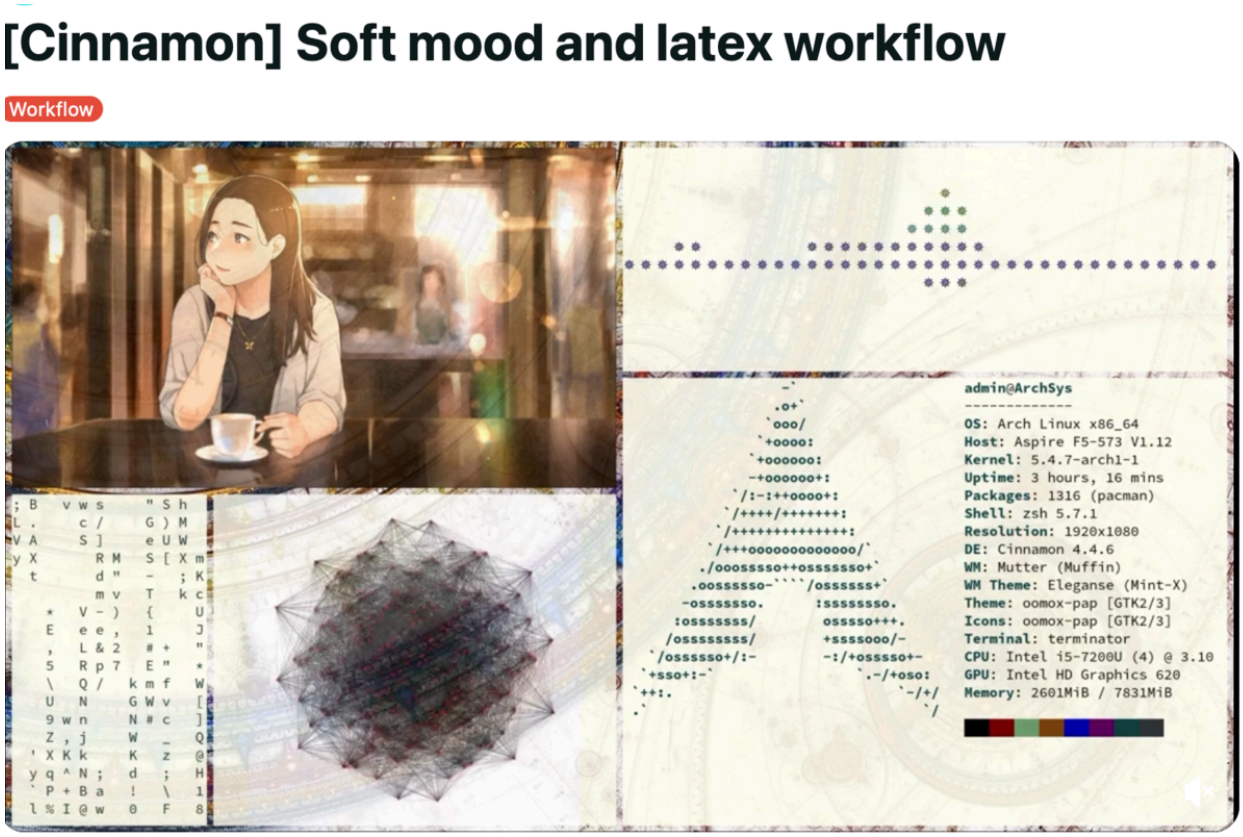

Figure 1: A close-up of a keyboard and terminal screen showing configuration files being edited.

The iterative process of refining a configuration file one setting at a time.

5 The Complete .vimrc Configuration

The following is the full `~/.vimrc` file. It is organised into sections: plugin declarations with vim-plug, plugin-specific settings inline with each `Plug` call, and global editor settings at the end.

```
set runtimepath^=~/vimplugins
syntax enable
filetype plugin indent on
let mapleader = ","
```

```
let maplocalleader = " "

call plug#begin('~/vimplugins')

" -- Colour schemes --
Plug 'rakr/vim-one'
Plug 'mhartington/oceanic-next'
Plug 'rafi/awesome-vim-colorschemes'
Plug 'tpope/vim-vividchalk'

" -- Clipboard --
Plug 'jasonccox/vim-wayland-clipboard'

" -- Alignment --
Plug 'junegunn/vim-easy-align'

" -- Undo tree --
Plug 'mbbill/undotree'

" -- Terminal REPL integration --
Plug 'jpalardy/vim-slime'

" -- LaTeX --
Plug 'lervag/vimtex'
let g:vimtex_complete_close_braces=1
let g:vimtex_quickfix_mode=0

" -- Snippets --
Plug 'sirver/ultisnips'
let g:UltiSnipsExpandTrigger="<C-tab>"
let g:UltiSnipsJumpForwardTrigger="<tab>"
let g:UltiSnipsJumpBackwardTrigger="<S-tab>"
nnoremap <leader>U \
  <Cmd>call UltiSnips#RefreshSnippets()<CR>
nnoremap <leader>u :UltiSnipsEdit<cr>

" -- Linting and fixing --
Plug 'dense-analysis/ale'
let g:ale_virtualtext_cursor = 'disabled'
let g:ale_set_balloons = 1
highlight clear ALEErrorSign
highlight clear ALEWarningSign
let g:ale_sign_error = '●'
let g:ale_sign_warning = '.'
let g:ale_linters = {
\   'python': ['pylsp']
```

```
\ }
let g:ale_fix_on_save = 1
let g:ale_fixers = {
\   '*': ['remove_trailing_lines',
\         'trim_whitespace'],
\   'r': ['styler'],
\   'rmd': ['styler'],
\   'quarto': ['styler'],
\   'python': ['black','isort'],
\   'javascript': ['eslint'],
\ }
nnoremap <leader>n :ALENext<CR>

" -- Fuzzy finder --
Plug 'junegunn/fzf',
  \ { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'
nnoremap <leader>z :Files<CR>
nnoremap <Leader>' :Marks<CR>
nnoremap <Leader>/ :BLines<CR>
nnoremap <Leader>b :Buffers<CR>
nnoremap <Leader>r :Rg<CR>
nnoremap <Leader>s :Snippets<CR>
tnoremap <leader>b <C-w>:Buffers<cr>
tnoremap <leader>r <C-w>:Rg<cr>
tnoremap <leader>z <C-w>:Files<cr>

" -- Navigation and editing --
Plug 'junegunn/vim-peekaboo'
Plug 'tpope/vim-unimpaired'
Plug 'tpope/vim-obsession'
Plug 'tpope/vim-repeat'
Plug 'tpope/vim-commentary'
autocmd FileType quarto
  \ setlocal commentstring=#\ %s
Plug 'tpope/vim-surround'
Plug 'justinmk/vim-sneak'
let g:sneak#label = 1
let g:sneak#s_next = 1
let g:sneak#use_ic_scs = 1
Plug 'machakann/vim-highlightedyank'

" -- Status line --
Plug 'vim-airline/vim-airline'
let g:airline#extensions#ale#enabled = 1
let g:airline#extensions#tabline#enabled = 1
```

```
let g:airline#extensions#fzf#enabled = 1

" -- R integration --
Plug 'rgt47/rgt-R'

" -- Completion --
Plug 'girishji/vimcomplete'

call plug#end()

" -- Editor settings --
if $COLORTERM == 'truecolor'
  set termguicolors
endif
set scrolloff=7
set iskeyword-=.
set completeopt=menu,menuone,popup,
  \noinsert,noselect
set complete+=k
set dictionary=/usr/share/dict/words
highlight Pmenu guifg=Black guibg=cyan
  \ gui=bold
highlight PmenuSel gui=bold guifg=White
  \ guibg=blue
set gfn=Monaco:h14
set encoding=utf-8
set lazyredraw
set autochdir
set number relativenumber
set clipboard=unnamed
set textwidth=80
set colorcolumn=80
set cursorline
set hlsearch
set incsearch
set ignorecase
set smartcase
set showmatch
set noswapfile
set hidden
set gdefault
set splitright
set wildmenu
set wildignorecase
set wildmode=list:full
```

```
" -- Popup menu navigation with Tab --
inoremap <expr> <tab>
  \ pumvisible() ? "\<C-n>" : "\<tab>"
inoremap <expr> <S-tab>
  \ pumvisible() ? "\<C-p>" : "\<S-tab>"

" -- Buffer and window navigation --
nnoremap <leader>o <C-w>:b1<CR>
nnoremap <leader>t <C-w>:b2<CR>
nnoremap <leader>h <C-w>:b3<CR>
nnoremap <leader><leader> <C-w>w
nnoremap <leader>a ggVG
nnoremap <leader>m vipgq
nnoremap <leader>f :tab split<cr>
nnoremap <leader>v :edit ~/.vimrc<cr>
nnoremap <localleader><leader> <C-u>
nnoremap <localleader><localleader> <C-d>
noremap - $
noremap : ;
noremap ; :
inoremap <F10> <C-x><C-k>
inoremap <F12> <C-x><C-o>
inoremap <silent> <Esc> <Esc>`^

" -- Terminal mode navigation --
tnoremap <F1> <C-\><C-n>
tnoremap <leader>o <C-w>:b1<CR>
tnoremap <leader>t <C-w>:b2<CR>
tnoremap <leader>h <C-w>:b3<CR>
tnoremap <leader><leader> <C-w>w
tnoremap lf ls()<CR>

" -- Auto-save and formatting --
au FocusGained * :let @z=@*
set updatetime=1000
autocmd CursorHold,CursorHoldI * update
autocmd FileType rmd
  \ setlocal commentstring=#\ %s
let $FZF_DEFAULT_OPTS =
  \ '--bind "ctrl-j:down,ctrl-k:up,'
  \ . 'j:preview-down,k:preview-up"'
set formatoptions-=c
  \ formatoptions-=r formatoptions-=o

" -- EasyAlign --
xmap ga <Plug>(EasyAlign)
```

```
nmap ga <Plug>(EasyAlign)

colorscheme one
set background=dark
```

After saving this file, open Vim and run:

```
:PlugInstall
```

Vim-plug will download and install every plugin declared between `plug#begin()` and `plug#end()`.

6 Filetype Plugin for R Markdown

Create the file `~/.vim/ftplugin/rmd.vim`. This file is loaded automatically whenever Vim opens a file with the `.Rmd` or `.rmd` extension. It defines terminal-mode shortcuts for rendering and sourcing, and normal-mode shortcuts for inspecting R objects under the cursor.

```
" ~/.vim/ftplugin/rmd.vim

" -- Terminal shortcuts for R Markdown --
tnoremap ZD quarto::quarto_render(
  \output_format = "pdf")<CR>
tnoremap ZO source("<C-W>"%")
tnoremap ZQ q('no')<C-\><C-n>:q!<CR>
tnoremap ZR render("<C-W>"%")<CR>
tnoremap ZT :!R -e 'render("<C-r>%",
  \ output_format="pdf_document")'<CR>
tnoremap ZS style_dir()<CR>
tnoremap ZX exit<CR>
tnoremap ZZ q('no')<C-\><C-n>:q!<CR>

" -- Object inspection under cursor --
nnoremap <localleader>d
  \ :let @c=expand("<cword>") \|
  \ :let @d="dim(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>h
  \ :let @c=expand("<cword>") \|
  \ :let @d="head(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>s
  \ :let @c=expand("<cword>") \|
  \ :let @d="str(".@c.")"."\n" \|
```

```
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>p
  \ :let @c=expand("<cword>") \|
  \ :let @d="print(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>n
  \ :let @c=expand("<cword>") \|
  \ :let @d="names(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>

" -- Auto-open R terminal --
autocmd BufEnter *
  \ if &ft ==# 'rmd' && !exists('b:entered')
  \ | execute(
  \   'let b:entered = 1 | :ter ++rows=5 R')
  \ | endif
```

The object inspection mappings deserve explanation. When the cursor is on a variable name, pressing `<localleader>d` extracts the word under the cursor, wraps it in `dim()`, and sends the command to the R terminal. This allows checking dimensions, printing heads, inspecting structure, and viewing names of any R object without leaving normal mode.

7 Filetype Plugin for R Scripts

Create the file `~/.vim/ftplugin/r.vim`. This file is nearly identical to the R Markdown ftplugin but uses the alternate file register (`#` instead of `%`) for sourcing commands, which is appropriate when the R script is not the current buffer.

```
" ~/.vim/ftplugin/r.vim

" -- Terminal shortcuts for R scripts --
tnoremap ZD quarto::quarto_render(
  \output_format = "pdf")<CR>
tnoremap ZO source("<C-W>"#")
tnoremap ZQ q('no')<C-\><C-n>:q!<CR>
tnoremap ZR render("<C-W>"#")
tnoremap ZS style_dir()<CR>
tnoremap ZX exit<CR>
tnoremap ZZ q('no')<C-\><C-n>:q!<CR>

" -- Object inspection under cursor --
nnoremap <localleader>d
```

```
  \ :let @c=expand("<cword>") \|
  \ :let @d="dim(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>h
  \ :let @c=expand("<cword>") \|
  \ :let @d="head(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>s
  \ :let @c=expand("<cword>") \|
  \ :let @d="str(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>p
  \ :let @c=expand("<cword>") \|
  \ :let @d="print(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>
nnoremap <localleader>n
  \ :let @c=expand("<cword>") \|
  \ :let @d="names(".@c.")"."\n" \|
  \ :call term_sendkeys(
  \   term_list()[0], @d)<CR>

" -- Auto-open R terminal --
autocmd BufEnter *
  \ if &ft ==# 'r' && !exists('b:entered')
  \ | execute(
  \   'let b:entered = 1 | :ter ++rows=5 R')
  \ | endif
```

8 UltiSnips: Static Snippets

UltiSnips is a snippet engine for Vim that expands trigger words into multi-line text templates. Each snippet definition lives in a filetype-specific file under `~/.vim/UltiSnips/`, for example `~/.vim/UltiSnips/rmd.snippets` for R Markdown files. A snippet consists of a trigger keyword, one or more tabstops (marked `$1`, `$2`, …) where the cursor will land after expansion, and the body text placed between those tabstops.

The critical configuration detail for this setup is avoiding conflicts between UltiSnips and popup menu navigation, since both want to use the Tab key. The solution is to assign `Ctrl-Tab` as the expand trigger and reserve plain `Tab` and `Shift-Tab` for jumping between tabstops. Separately, the popup menu mappings in `.vimrc` use `Tab` and `Shift-Tab` only when a popup is visible.

```
let g:UltiSnipsExpandTrigger="<C-tab>"
let g:UltiSnipsJumpForwardTrigger="<tab>"
let g:UltiSnipsJumpBackwardTrigger="<S-tab>"

inoremap <expr> <tab>
  \ pumvisible() ? "\<C-n>" : "\<tab>"
inoremap <expr> <S-tab>
  \ pumvisible() ? "\<C-p>" : "\<S-tab>"
```

With this configuration, the workflow is:

1. Type a snippet trigger word (e.g., `rheader`).
2. Press `Ctrl-Tab` to expand the snippet.
3. Press `Tab` to jump to the next tabstop.
4. When a popup menu appears during typing, `Tab` navigates forward through completions instead.

A minimal R Markdown YAML header snippet looks like this in the snippets file:

```
snippet rheader "R Markdown YAML header"
---
title: "$1"
author: "${2:R.G. Thomas}"
date: today
output:
  pdf_document:
    keep_tex: true
header-includes:
  - \usepackage{lipsum, fancyhdr,
      titling, currfile}
  - \usepackage[export]{adjustbox}
  - \pagestyle{fancy}
---
$0
endsnippet
```

When `rheader` is typed and `Ctrl-Tab` is pressed, the template expands and the cursor lands at `$1` (the title field). Pressing `Tab` moves to `$2` (the author field, which has a default value), and then to `$0` (the final resting position after all tabstops are filled).

9 UltiSnips: Python-Interpolated Dynamic Snippets

Static snippets cover predictable boilerplate. There is a class of problem, however, where the expansion content must be computed at the moment of triggering based on something the user has typed. UltiSnips supports three interpolation mechanisms beyond static text: shell execution (backtick), Vimscript (`!v`), and Python (`!p`). Python interpolation is the most capable: it executes

an arbitrary Python expression inside the snippet body and inserts the result into the expanded text.

The `!p` marker tells UltiSnips to treat the enclosed block as Python. Inside that block, the special variable `snip.rv` (rv standing for 'return value') holds whatever the snippet should insert at that position. Any valid Python expression can compute `snip.rv`, including imports, loops, and string formatting.

9.1 A Concrete Motivating Problem

A question on Stack Overflow (https://stackoverflow.com/questions/78636197) asked how to write a snippet that generates a variable number of chord placeholders. The number of placeholders depends on a value the user types into the first tabstop. This is exactly the kind of problem that static snippets cannot solve: the body cannot be known at the time the snippet file is written because it depends on runtime input.

The Python function that solves this problem is straightforward:

```
   def make_chords(num_reps):
       text = ""
       for i in range(num_reps):
           text += "< > "
       return text
```

To embed this inside an UltiSnips snippet definition:

```
snippet chords "Generate chord placeholders"
`!p
def make_chords(num_reps):
    text = ""
    for i in range(num_reps):
        text += "< > "
    return text

snip.rv = make_chords(int(t[1]))
`
endsnippet
```

The backtick delimiters with `!p` open and close the Python block. Inside the block, `t[1]` refers to the current value of tabstop 1. When the user types a number into the first tabstop and moves focus, UltiSnips re-evaluates the Python block and replaces the output accordingly. The `int()` conversion is required because `t[1]` is always a string.

9.2 Inserting the Current Date

A simpler but commonly useful dynamic snippet inserts the current date at expansion time:

```
snippet today "Insert today's date"
`!p
import datetime
snip.rv = datetime.date.today().isoformat()
`
endsnippet
```

This snippet takes no tabstops. When triggered, it imports the `datetime` module and assigns the ISO-formatted date string to `snip.rv`. The result is a plain text date that does not change after insertion, unlike a live template that re-evaluates on every keystroke.

9.3 A Numbered List Generator

A more elaborate pattern uses Python to generate a numbered list of a length specified by the user:

```
snippet numlist "Numbered list of N items"
${1:5}
`!p
n = int(t[1])
snip.rv = "\n".join(
    f"{i+1}. " for i in range(n)
)
`
endsnippet
```

When `5` is the value of tabstop 1, the expansion becomes five numbered lines. Changing the number before moving off the tabstop regenerates the list.

9.4 Debugging Python Snippets

When a Python interpolation block fails silently, the most reliable diagnostic is to open Vim's message log with `:messages` immediately after a failed expansion. Python exceptions inside UltiSnips are caught and displayed as Vim error messages. A second approach is to test the Python logic independently in a Python interpreter before embedding it, since the UltiSnips execution environment is standard CPython with no special restrictions beyond access to the `snip` object.

```
" Force UltiSnips to re-read all snippet files
:call UltiSnips#RefreshSnippets()

" Open the snippet file for the current filetype
```

```
:UltiSnipsEdit
```

If the expansion trigger fires but produces no output, confirm Python 3 support with `:echo has('python3')`. A return value of `0` means the Vim binary was compiled without Python and the entire `!p` mechanism is unavailable.

10 Practical Application

This section walks through a concrete example of using the configured environment. The goal is to perform a logistic regression on the Palmer Penguins dataset, predicting gender, using only Vim and the R terminal.

Step 1: Create a working directory and open an empty R Markdown file.

```
cd ~/prj/penguins
vim -u ~/.vimrc p.Rmd
```

Step 2: Enter insert mode and type the first R command to load the data.

```
i
library(palmerpenguins)
```

Step 3: Exit insert mode with `Ctrl-C`, then yank the line into the unnamed register.

```
<C-c>
yy
```

Step 4: The ftplugin for R Markdown automatically opens an R terminal in a split pane. If it has not opened yet, use `:ter R` to start one manually.

Step 5: In the terminal pane, paste the yanked line from the register:

```
<C-w>""
```

This sends `library(palmerpenguins)` to the R REPL and executes it.

Step 6: Use UltiSnips to scaffold the R Markdown header. Type `rheader` on the first line and press `Ctrl-Tab`. The snippet expands into a YAML header template with tabstops for the project name, title, author, and bibliography path. Press `Tab` to navigate between tabstops. Do not leave insert mode while navigating or the snippet session will end.

11 R Markdown Template Structure

For reference, a minimal R Markdown YAML header for PDF output with LaTeX customisation looks like this:

```
---
title: "Penguins data analysis"
author: "R.G. Thomas"
date: today
output:
  pdf_document:
    keep_tex: true
header-includes:
  - \usepackage{lipsum, fancyhdr,
      titling, currfile}
  - \usepackage[export]{adjustbox}
  - \pagestyle{fancy}
---
```

A useful tip for file completion in Vim when editing R Markdown or Quarto files: temporarily set the filetype to `tex` with `:set filetype=tex`, then type `\includegraphics{` or `\input{` followed by `Ctrl-X Ctrl-O` for omni-completion of file paths.

11.1 Things to Watch Out For

1. **Tab key conflicts.** UltiSnips and popup menu completion both use Tab. If snippets stop expanding, check that `g:UltiSnipsExpandTrigger` is set to `<C-tab>` rather than `<tab>`.
2. **ftplugin not loading.** If the R terminal does not open automatically, verify that `filetype plugin indent on` appears in the `.vimrc` before any `Plug` calls, and that the ftplugin files are in `~/.vim/ftplugin/` (not `~/.vim/plugin/`).
3. **ALE styler dependency.** The ALE fixer for R uses the `styler` package. If ALE reports errors on save, install styler in R first: `install.packages("styler")`.
4. **Colour scheme not found.** If Vim reports `E185: Cannot find color scheme`, run `:PlugInstall` to ensure the colour scheme plugin has been downloaded.
5. **Terminal register paste.** The `<C-w>""` paste in terminal mode pastes from the unnamed register. If the text was yanked into a named register, use `<C-w>"a` (for register `a`).
6. **Python interpolation requires Python 3 support.** Vim must be compiled with `+python3`. The default macOS system Vim often lacks this. If `:echo has('python3')` returns `0`, install Vim via Homebrew (`brew install vim`) or use Neovim with a Lua snippet engine such as LuaSnip.
7. **Leaving insert mode terminates the snippet session.** During tabstop navigation, pressing `Escape` ends the session and leaves the remaining tabstop markers as literal text. If this happens, undo the expansion with `u` and re-trigger the snippet.

Figure 2: A calm library reading room with natural light filtering through tall windows.

Stepping back to reflect on what was learnt.

11.2 Lessons Learnt

11.2.1 Conceptual Understanding

- A Vim-based R and LaTeX workflow replaces application switching with buffer switching, which is measurably faster once muscle memory develops.
- Filetype plugins are the correct mechanism for language-specific settings in Vim; global .vimrc settings should remain language-agnostic.
- The separation between snippet expansion (Ctrl-Tab) and popup navigation (Tab) is essential for avoiding conflicts in a multi-plugin environment.
- UltiSnips tabstops provide a structured way to scaffold repetitive code patterns, reducing boilerplate errors in YAML headers.
- Static snippets and Python-interpolated snippets occupy different parts of the solution space: use static snippets for fixed structure and Python snippets when the expansion depends on runtime input.

11.2.2 Technical Skills

- Configuring vim-plug with inline `let` statements keeps plugin settings co-located with their declarations, improving maintainability.
- The `term_sendkeys()` function in Vim 8+ enables programmatic communication with terminal buffers, which is the foundation for REPL integration.
- ALE fixers configured per filetype (`r`, `rmd`, `quarto`) apply consistent code style without manual intervention.
- The `autocmd BufEnter` pattern with a guard variable (`b:entered`) prevents multiple R terminals from spawning when switching buffers.
- The `!p` marker in UltiSnips gives access to a full Python environment; `snip.rv` is the single output variable that carries the computed text back into the expansion.
- `t[1]` inside a Python block reads the current value of tabstop 1, enabling parametric expansions where the output depends on what the user has typed.

11.2.3 Gotchas and Pitfalls

- Leaving insert mode during UltiSnips tabstop navigation silently terminates the snippet session, leaving partially expanded text.
- The `autochdir` setting can interfere with relative file paths in R Markdown documents if the working directory changes unexpectedly.
- ALE's `ale_fix_on_save` may conflict with files that are intentionally unformatted (e.g., raw data files opened in Vim).
- The `set iskeyword-=.` setting breaks word boundaries at dots, which is helpful for R (where `data.frame` is two words) but can confuse navigation in other filetypes.
- Python exceptions inside UltiSnips are swallowed silently unless `:messages` is checked; a snippet that produces no output is often a Python runtime error rather than a configuration problem.

11.3 Limitations

- This configuration is specific to Vim 8+ with terminal support; Neovim requires different terminal API calls (`jobsend` instead of `term_sendkeys`).
- The ftplugin auto-opens R in a 5-row terminal split, which may be too small on low-resolution displays.
- ALE's `styler` fixer requires the `styler` R package to be installed in the active R library; if missing, ALE silently fails to fix on save.
- UltiSnips requires Vim compiled with Python 3 support; the default macOS system Vim may lack this. Neovim users should use LuaSnip, which provides equivalent Python-like interpolation through Lua functions.
- Python code inside UltiSnips snippets runs synchronously on the main thread. Expensive computations (file reads, network requests) will block the editor for the duration.
- The configuration does not include debugging support; stepping through R code requires a separate tool or plugin.
- Vimtex's forward and inverse search with a PDF viewer (e.g., Zathura or Skim) requires additional configuration not covered here.

- Python snippet code runs in a shared interpreter state; variables defined in one snippet block persist into subsequent expansions in the same session, which can produce unexpected results if variable names collide.

11.4 Opportunities for Improvement

1. Add Neovim-compatible terminal integration using `vim.fn.jobsend()` for cross-editor portability.
2. Configure vimtex forward search with Skim or Zathura for real-time PDF preview during LaTeX editing.
3. Create project-specific `.vimrc` files that override the terminal split size based on the display resolution.
4. Develop additional UltiSnips templates for common R analysis patterns (ggplot scaffolds, model fitting boilerplate, knitr chunk options).
5. Port the Python interpolation patterns to LuaSnip for Neovim users, using Lua functions in place of Python `!p` blocks.
6. Integrate vim-slime as an alternative to the built-in terminal for sending code to tmux sessions.
7. Add a Quarto-specific ftplugin file for `.qmd` files with render commands that target both HTML and PDF output.

12 Wrapping Up

The configuration described in this post transforms Vim into a unified environment for R programming and LaTeX document preparation, then extends it with programmable snippet expansion through Python interpolation. The core insight is that filetype plugins and a carefully managed Tab key mapping are the two pillars of the static setup, while `snip.rv` and the `t[]` tabstop array are the two primitives needed for dynamic expansions.

What I learnt most from this process is that configuration is iterative. The .vimrc I use today looks nothing like the one I started with. Each problem encountered (a Tab conflict, a missing colour scheme, a terminal that would not open) taught me something about how Vim dispatches events and how plugins interact. The Python interpolation extension followed the same pattern: a concrete problem (a Stack Overflow question about chord placeholders) revealed a capability I had not known existed.

For anyone attempting a similar setup, start with just vimtex and one ftplugin file. Get the R terminal auto-opening reliably before adding UltiSnips or ALE. Add static snippets next. Reach for Python interpolation only after encountering a problem that static snippets genuinely cannot solve. Layering complexity gradually makes debugging much simpler.

In conclusion, five points merit emphasis. First, use `Ctrl-Tab` for snippet expansion to avoid Tab key conflicts with popup menu navigation. Second, place filetype-specific settings in `~/.vim/ftplugin/` rather than cluttering the global .vimrc. Third, ALE with `styler` provides automatic R code formatting on save with no manual intervention. Fourth, the `term_sendkeys()` function enables sending arbitrary R commands from normal mode to a running R REPL. Fifth, the `!p` marker in UltiSnips enables Python interpolation: assign the result to `snip.rv` and read tabstop values from `t[1]`, `t[2]`, etc.

13 See Also

Related posts:

- 'Setting up a minimal Neovim configuration for data science' (companion post on Neovim plugin installation)
- 'Configure the Command Line for Data Science Development' (terminal and shell configuration)
- Post 26: Setting up Neovim
- Post 52: The Workflow Construct

Key resources:

- vimtex documentation
- UltiSnips repository
- UltiSnips full documentation
- ALE (Asynchronous Lint Engine)
- vim-plug plugin manager
- Vimtex and Zathura setup guide
- UltiSnips Python interpolation (Vimcasts)
- LuaSnip (Neovim alternative)
- Stack Overflow: chord placeholder snippet
- Latexmk documentation

14 Reproducibility

This configuration was developed and tested on macOS with Vim 9.0, Python 3.11, and R 4.4. The following files constitute the complete configuration:

File	Purpose
`~/.vimrc`	Global plugin and editor settings
`~/.vim/ftplugin/rmd.vim`	R Markdown keybindings
`~/.vim/ftplugin/r.vim`	R script keybindings
`~/.vim/UltiSnips/rmd.snippets`	R Markdown snippet library
`~/.vim/UltiSnips/all.snippets`	Filetype-agnostic snippets

To reproduce the environment:

```
curl -fLo ~/.vim/autoload/plug.vim \
  --create-dirs \
  https://raw.githubusercontent.com/\
junegunn/vim-plug/master/plug.vim

cp vimrc ~/.vimrc
cp ftplugin/rmd.vim ~/.vim/ftplugin/rmd.vim
cp ftplugin/r.vim ~/.vim/ftplugin/r.vim
```

```
vim +PlugInstall +qall

Rscript -e "install.packages('styler')"
```

To verify Python 3 support after installation:

```
:echo has('python3')
:echo g:UltiSnipsExpandTrigger
```

15 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.

Rendered on 2026-05-17 at 17:08 PDT. Source: ~/prj/qblog/posts/30-setupRvimtex/setupRvimtex/analysis/report/

15.1 Related posts in this cluster

This post is part of the *Workflow Construct* series. Recommended reading order:

1. Post 15: A Workflow Construct for the Modern Data Scientist
2. Post 16: Unix Command-Line Workspace Setup for Data Science
3. Post 17: Multi-Laptop macOS Bootstrap
4. Post 18: Setting Up Git for Data Science Workflows
5. Post 19: Setting Up Neovim as a Data Science IDE
6. **Post 20: Extending the R-Vim Workflow with LaTeX** (this post)
7. Post 21: Modern CLI Replacements for the Shell Layer
8. Post 22: LLM-Augmented Editing for the Workflow Construct
9. Post 23: Configuring Yabai as a Tiling Window Manager
10. Post 24: A pocket terminal with ttyd and Tailscale
11. Post 25: Install Linux Mint on a MacBook Air